

of data to be changed in field/value pairs, and the third argument is an array of field/value pairs used to provide “where” conditions for the update. All three of these arguments are required.

The first two arguments are identical to the insert() method. The fourth and fifth arguments, respectively, behave similarly to the format argument in insert(). The fourth argument is an array of data formats (%s for string and %d for number) that forces the data that will be inserted into the database to be treated as the appropriate data type. The fifth argument is identical, except it pertains to the data used in the WHERE clause.

Using the update() method to safely update data in MySQL

```
function edit_music_post( $post_id, $new_title, $new_
description )
{
global $wpdb;
$wpdb->update( $wpdb->posts, array( 'post_title' => $new_
title, 'post_
content' => $new_content ), array( 'ID' => $post_id ),
array( '%s',
'%s'), array( '%d' ) );
}
```

8.5 Retrieving data from MySQL with WordPress

It's great that the database class has so many properties but what good is a Web application if it can't read and write to a database? Of course, WordPress can and does. A number of methods exist that will retrieve data in an elegant, object-oriented way if necessary. It is always good practice to avoid having to include raw SQL in a plugin when possible. The reality is that plugins that rely on raw SQL are often the ones that break later on. A good example of this is the myriad of plugins that used the categories table prior to WordPress 2.3. When the \$wpdb->categories and \$wpdb->post2cat tables disappeared and were replaced by the new terms paradigm, the plugins that were querying the categories tables directly broke. In many cases, WordPress provides methods and functions to retrieve data from MySQL as part of its plugin API. Where this API exists, you should use it because it is generally forward compatible.

Retrieving a single value from a table

In the event that an API doesn't exist to retrieve the data needed, you can use